Verifying and Testing Cloud Network Connectivity, Security, and Data Protection in AWS Tower

Deepika Agnihotri

CLCS 635 Networking Engineering for Cloud Computing

April 28, 2026

Introduction:

Cloud networking is a critical component of modern cloud computing environments because it enables secure communication between resources, users, and applications hosted in the cloud. Proper network configuration ensures that cloud resources remain accessible, scalable, and protected from unauthorized access. In AWS, configuring VPCs, subnets, routing tables, security groups, and network access control lists is essential for maintaining secure and efficient communication between cloud resources.
The primary objective of this lab was to provision, configure, and validate a secure cloud networking environment using AWS. The lab required deploying two EC2 instances--one in a public subnet and one in a private subnet--using both the AWS Management Console and AWS Application Programming Interfaces. Secure Shell connectivity and network testing were then used to verify that the public instance could access the internet, while the private instance remained inaccessible from the internet and could only be reached through the public instance.

This lab also emphasized the importance of infrastructure automation using Python and AWS APIs. By automating the deployment of instances, the lab demonstrated how cloud administrators and Site Reliability Engineers can improve efficiency, reduce human error, and standardize deployments across cloud environments.

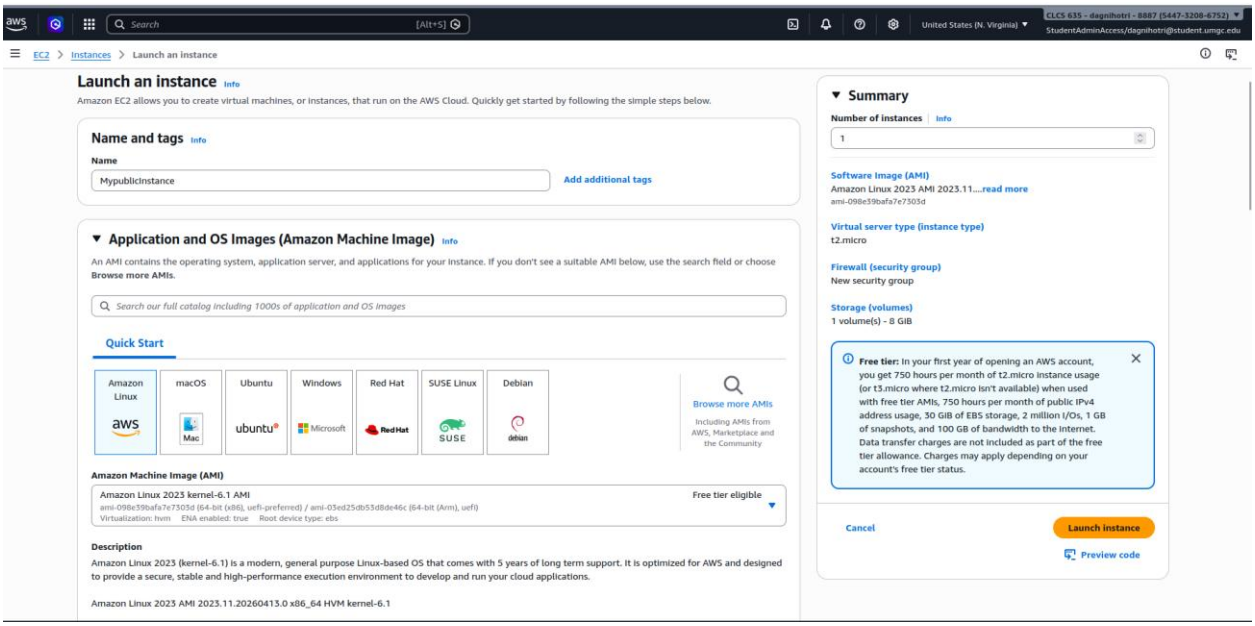
Methodology:

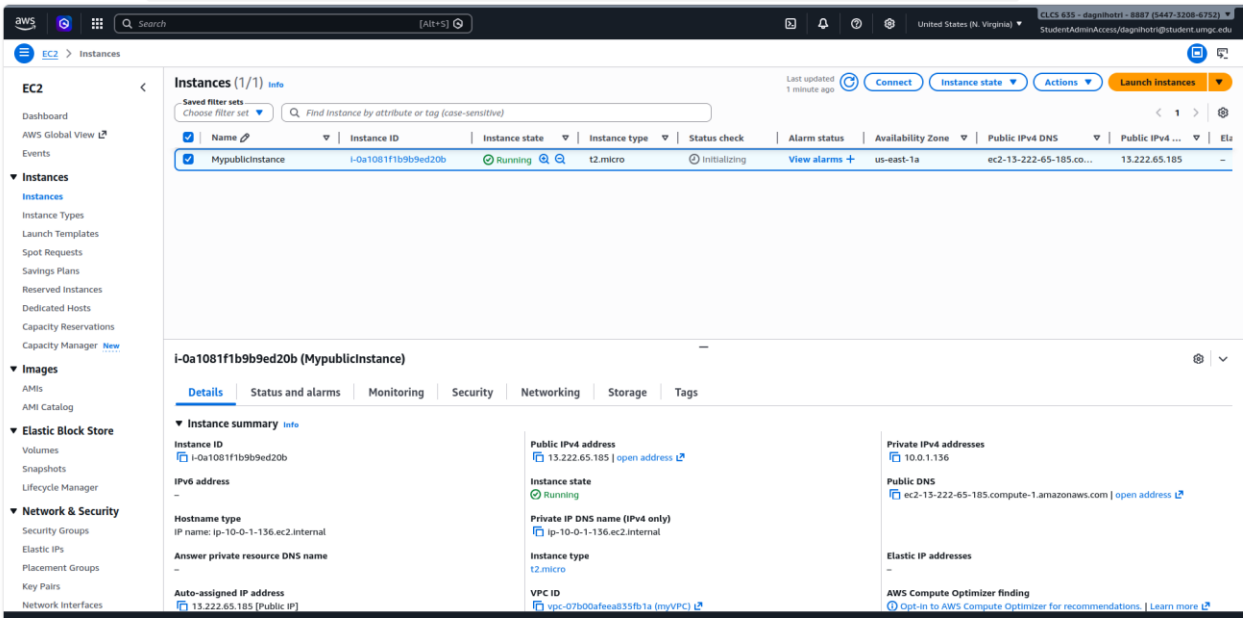
Environment Setup:

The lab environment was created within AWS using a VPC architecture that included both public and private subnets. The public subnet was configured to allow internet access through an Internet Gateway, while the private subnet was isolated from direct internet access to improve security.
Below AWS services and tools were used during the lab:
Amazon EC2 - Used to provision virtual machine instances.
Amazon VPC - Used to create isolated networking environments.
Security Groups - Configured as virtual firewalls to control inbound and outbound traffic. Route Tables - Configured to control traffic routing between subnets and gateways.
AWS CloudShell - Used to execute Python scripts directly in the AWS environment.
Python with Boto3 SDK - Used to automate EC2 instance creation.

Step 1: Provision an AWS instance in public subnet, create a security group using AWS GUI

The first step involved creating an EC2 instance within the public subnet using the AWS Management Console. A security group was configured to allow secure inbound access for administration and connectivity testing. The instance was associated with a public IP address to allow external connectivity.





This screenshot demonstrates the creation of the public EC2 instance and the associated security group configuration through the AWS graphical user interface.

Step 2: Analyze and Edit create_instance.py Python script using AWS Python API call to provision your private subnet instance

The next step involved analyzing and modifying the Python automation script named Unit 7 create_instance.py. The script was updated to provision an EC2 instance within the private subnet instead of the public subnet. Modifications included updating subnet IDs, security group IDs, and instance configuration parameters.

The script utilized the AWS Boto3 SDK to automate the provisioning process. Using automation improved consistency and reduced the need for repetitive manual configuration.

```
def main():

    ec2_client = boto3.client('ec2')

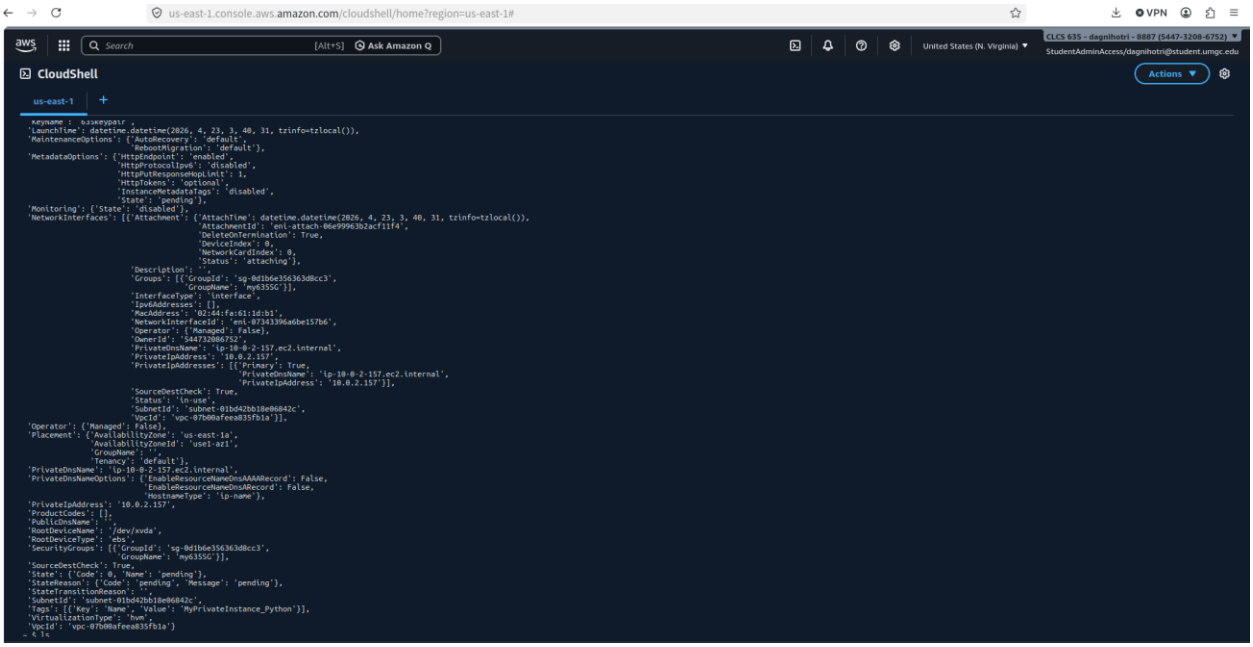

    image_id = 'ami-0d05471b100e9083f'
    instance_type = 't2.micro'
    keypair_name = '635keypair'
    security_group_ids = ['sg-0d1b6e356363d8cc3',]
    subnet_id = 'subnet-01bd42bb18e06842c'
```

This screenshot demonstrates the modifications made to the Python script to support deployment within the private subnet.

Step 3: Run the Python script in CloudShell

After updating the script, the Python file was executed within AWS CloudShell. The script successfully provisioned an EC2 instance inside the private subnet.

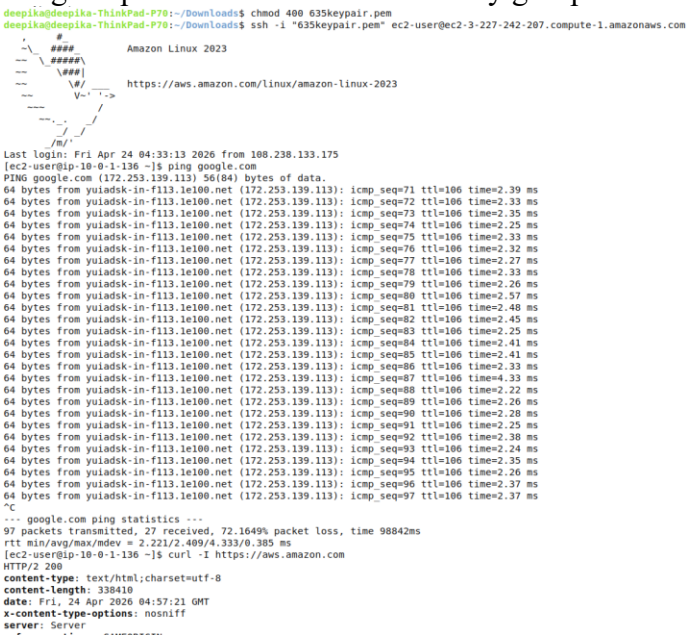
AWS CloudShell provides a browser-based command-line environment that simplifies interaction with AWS services and supports rapid execution of automation scripts.



This screenshot demonstrates successful execution of the automation script and creation of the private subnet instance.

Step 4: Verify Connectivity of your Public Instance

Connectivity testing was performed on the public instance to verify external accessibility. The tests confirmed that the public instance was reachable and properly configured through the assigned public IP address and security group rules.

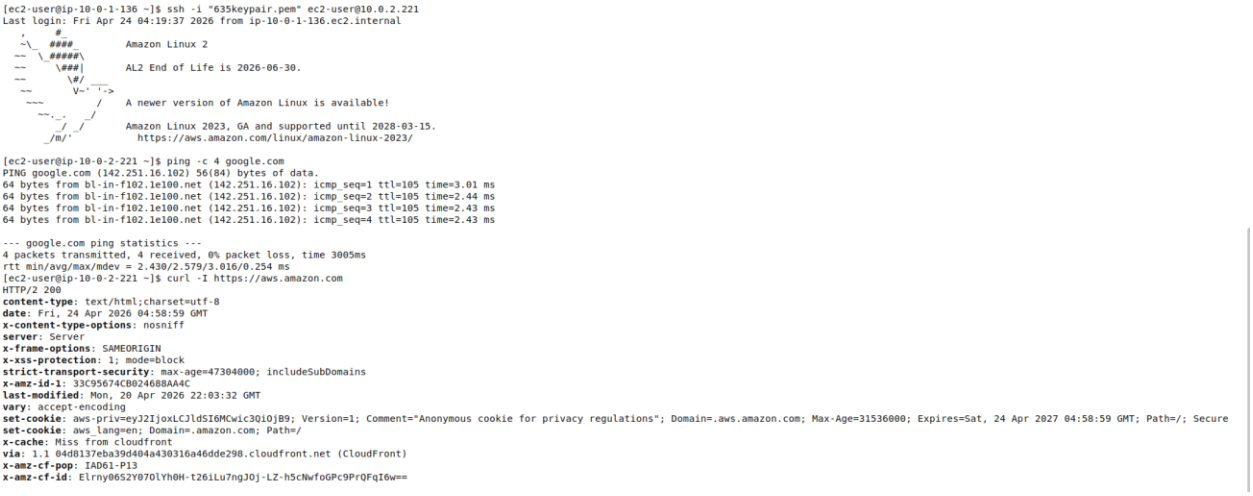


The image shows a successful verification of public EC2 instance connectivity. It documents securing the SSH key using chmod 400, followed by a successful SSH login to an Amazon Linux 2023 public EC2 instance, confirming that the instance is accessible from the internet. The execution of ping google.com demonstrates outbound network connectivity, while the curl -I https://aws.amazon.com command returns a valid HTTP 200 response, confirming HTTPS internet access. Overall, the image verifies that the public instance is correctly configured with proper routing, security group rules, and Internet Gateway access.

Step 5: Verify Connectivity of your Private Instance

Connectivity testing was also performed for the private instance. Since the instance resided in a private subnet, it was not directly accessible from the public internet. Access to the private instance required communication through authorized internal networking paths.

```
deepika@deepika-ThinkPad-P70:~/Downloads$ scp -i 635keypair.pem 635keypair.pem ec2-user@ec2-13-222-65-185.compute-1.amazonaws.com:/home/ec2-user/
635keypair.pem
deepika@deepika-ThinkPad-P70:~/Downloads$
```



The image shows a successful SSH connection from the public EC2 instance to the private EC2 instance using the private IP address, confirming that the private instance is accessible only from within the VPC. It demonstrates that the private instance is running Amazon Linux 2 and validates secure, internal access rather than direct internet access.

The image also verifies outbound internet connectivity from the private instance. The successful ping google.com results and the curl -I https://aws.amazon.com command returning an HTTP/2 200 response confirm that the private instance can reach external internet resources through the NAT Gateway, while remaining protected from inbound internet connections.

Results and Screenshots:

Public Instance Deployment Results

The public EC2 instance was successfully deployed within the public subnet. The instance received a public IP address and was reachable through the internet. Security group rules allow controlled inbound access for administrative connectivity.

The connectivity test confirmed that:

- The public subnet routing configuration was functioning correctly.
- The Internet Gateway was properly attached to the VPC.
- The security group rules permitted the required traffic.
- The instance was reachable from external networks.

These results verified that the public subnet configuration successfully enabled internet-facing communication.

Private Instance Deployment Results

The private EC2 instance was successfully provisioned through the Python automation script.

The instance operated within the private subnet and did not expose a public IP address.
The connectivity tests demonstrated that:

- The private subnet successfully restricted direct internet access.
- Internal communication between instances was properly configured.
- Security group settings allow controlled communication only from approved sources.
- The private subnet architecture improved isolation and reduced exposure to external threats.

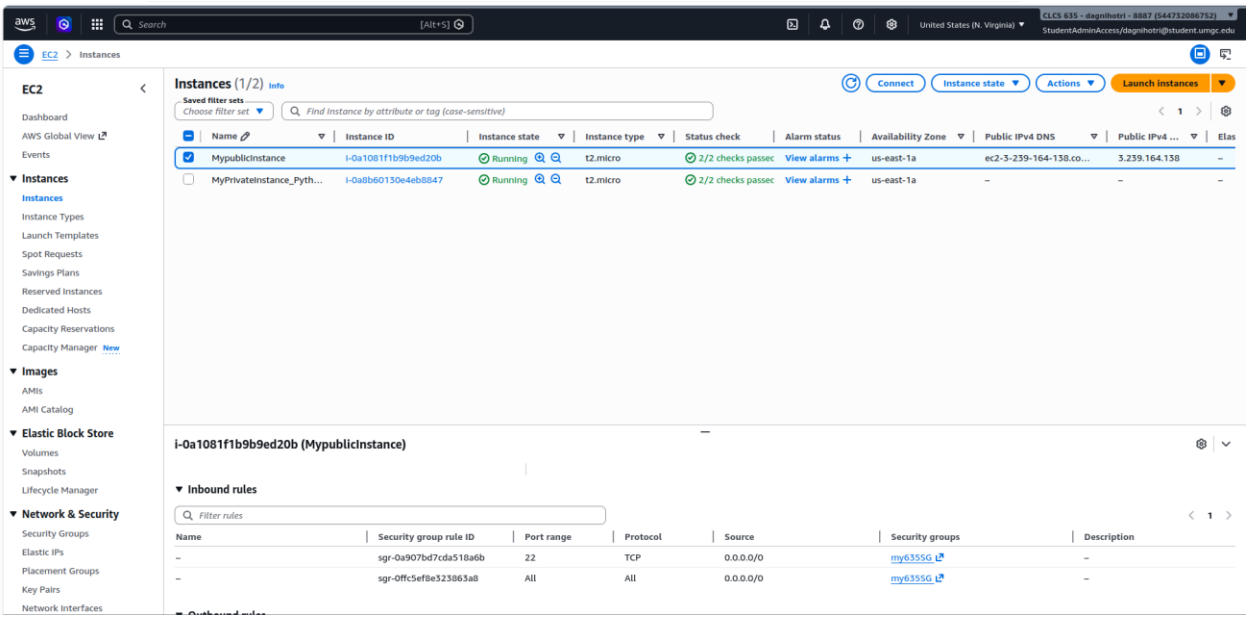
The results validated that the private subnet design effectively protected internal cloud resources.

Security Analysis:
Security Measures Implemented
Multiple security layers were implemented in this lab, including routing controls, subnet segmentation, and security group restrictions. Public subnets were configured to route internet-bound traffic (0.0.0.0/0) to an Internet Gateway, while private subnets routed internet-bound traffic exclusively to a NAT Gateway, preventing inbound internet access.

Role of Public and Private Subnets in Network Security
Public subnets are designed to host resources that must be accessible from the internet, such as bastion or jump hosts. In contrast, private subnets host internal resources that should not be directly reachable from external networks. This separation limits exposure and ensures sensitive workloads are protected from direct attacks.

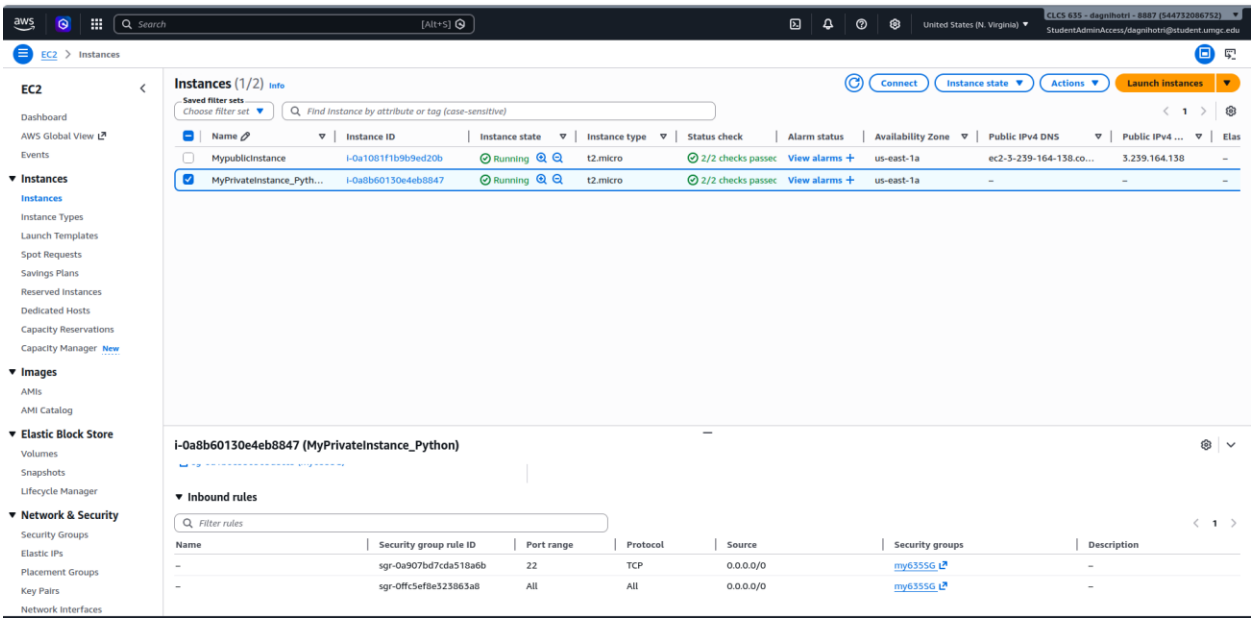
Security Groups
Security groups acted as virtual firewalls for the EC2 instances. Rules were configured to allow only authorized inbound and outbound traffic.

Public Instance Security Group
The public instance security group allows limited inbound access required for remote administration and testing.



This screenshot demonstrates the inbound and outbound rules configured for the public instance.

Private Instance Security Group
The private instance security group restricted inbound traffic to trusted internal sources only.



This screenshot demonstrates how access to the private instance was restricted to improving security.

Network ACLs:

Network Access Control Lists provide an additional layer of subnet-level security. Unlike security groups, which operate at the instance level, NACLs filter traffic entering and leaving subnets.

The use of NACLs strengthens security by:

Blocking unauthorized traffic at the subnet level.

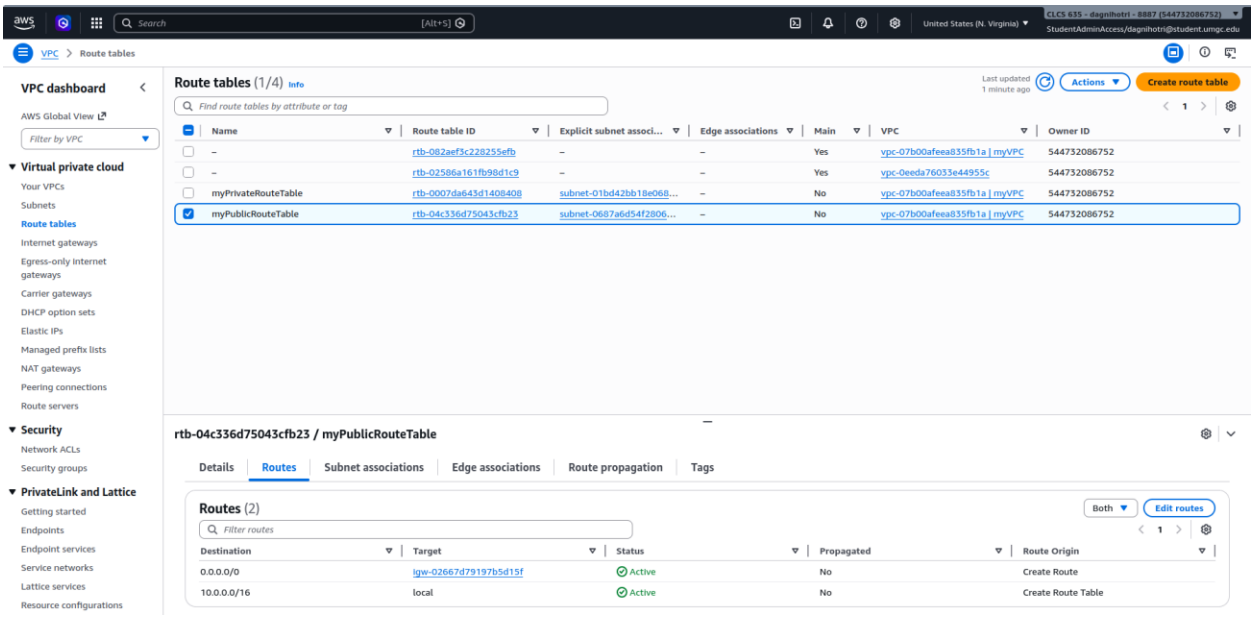
Supporting defense-in-depth security architecture.

Together, security groups and NACLs create multiple layers of protection for cloud resources.

Route Tables: Route tables controlled how traffic moved within the VPC.

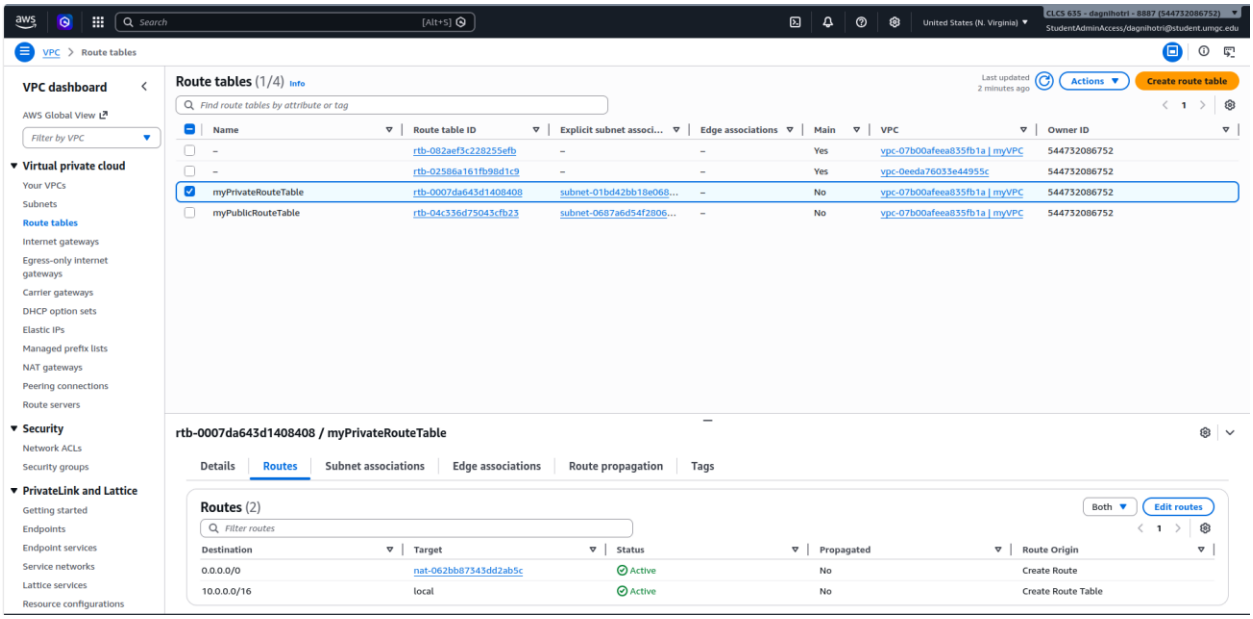
Public Route Table

The public route table includes a route to the Internet Gateway, allowing internet communication resources in the public subnet.



This screenshot demonstrates the routing configuration that enables internet connectivity for the public subnet.

Private Route Table: The private route table restricted direct internet access and controlled internal communication paths.



This screenshot demonstrates the routing configuration used to isolate the private subnet.

Challenges and Solutions

During the lab, several challenges were encountered while configuring the cloud networking environment and automating instance deployment. One of the primary challenges faced correctly configuring security group rules and routing tables to allow proper communication between

resources While maintaining security restrictions. Initially, connectivity tests failed because the security group rules did not allow the required inbound traffic for testing. This issue was resolved by carefully reviewing the inbound and outbound rules and updating them to permit only the necessary protocols
and ports.
Another challenge faced while modifying the Unit 7 create_instance.py script for deployment in the private subnet. Incorrect subnet IDs and security group configurations initially caused instance creation failures. Troubleshooting involved reviewing the AWS CloudShell output, verifying the subnet configuration, and validating the Boto3 API parameters. After correcting the subnet ID and security group for references, the script successfully provisioned the private EC2 instance.
Third challenge faced with an invalid AMI ID error during API-based provisioning of the private instance. This issue occurred because AMI IDs are region-specific and the original AMI value did not exist in the selected AWS region. The problem was resolved by identifying a valid Amazon Linux 2 AMI for the active region using the EC2 console and updating the Python script accordingly.

A further challenge was understanding the differences between public and private subnet routing behavior. The private instance could not directly access the internet due to the absence of an Internet Gateway route. This limitation reinforced the importance of proper network segmentation and routing design in secure cloud architectures.

Lessons Learned

This lab provides valuable practical experience in configuring secure cloud networking environments using AWS services. One of the key lessons learned was the importance of separating public-facing resources through the use of public and private subnets.
The lab also demonstrated how security groups and Network ACLs work together to provide layered security controls. Properly configured firewall rules are essential for controlling access to cloud resources and minimize exposure to potential threats.
Another important lesson involved the value of automation in cloud environments. Using Python and the AWS Boto3 SDK simplified infrastructure provisioning and demonstrated how automation improves consistency, scalability, and operational efficiency.
Additionally, the lab reinforced the importance of troubleshooting and validation during cloud deployments. Connectivity testing and route verification were critical for ensuring that the environment functioned correctly and securely.
These concepts directly apply to real-world cloud deployments where organizations must maintain secure, scalable, and highly available cloud infrastructure. Network segmentation, automated provisioning, and layered security controls are widely used best practices in enterprise cloud environment.

Conclusion

This lab successfully demonstrated the configuration and verification of secure cloud networking components within AWS. The activities included deploying EC2 instances in both public and private subnets, configuring security groups and route tables, modifying and executing automation scripts, and performing connectivity testing.
The results confirmed that the public subnet allowed controlled external access while the private subnet effectively restricted direct internet exposure. Security groups and Network ACLs

provided layered protection for cloud resources, and automation through Python scripting improved deployment efficiency.
Overall, the lab significantly enhanced understanding of cloud networking concepts, subnet segmentation, routing configuration, and cloud security practices. The hands-on experience gained through this exercise provided valuable insight into how secure and scalable cloud infrastructures are designed and managed in real-world environments.

Appendix:

```
import boto3

from botocore.exceptions import ClientError
import pprint
#It was modified to include the correct region-specific AMI ID, key pair, security group ID, and private subnet ID.

def main(): ec2_client = boto3.client('ec2')

    # Amazon Linux 2 AMI ID valid for the active AWS region
    image_id = 'ami-0d05471b100e9083f'

    # Instance type selected for low cost and lab requirements
    instance_type = 't2.micro'

    # Existing key pair used for SSH access (same as public instance)
    keypair_name = '635keypair'

    # Security group ID that allows SSH ONLY from the public instance
    security_group_ids = ['sg-0d1b6e356363d8cc3']

    # Private subnet ID where the instance is launched
    subnet_id = 'subnet-01bd42bb18e06842c'

  try:

      response = ec2_client.run_instances(
          ImageId=image_id,
          InstanceType=instance_type,
          KeyName=keypair_name,
          MinCount=1,
          MaxCount=1,
          SubnetId=subnet_id,
          SecurityGroupIds=security_group_ids,
          TagSpecifications=[
              {
                  'ResourceType': 'instance',
                  'Tags': [
```

```
                {
                    'Key': 'Name',
                    'Value': 'MyPrivateInstance_Python'
                }
            ]
        }
    ],
)

  # Print details of the created instance for verification
   pprint.pprint(response['Instances'][0])

except ClientError as e:
   # Print any errors encountered during instance creation
   print(e)


if name == 'main':

main()
```

Explanation of Script Modifications:

The following modifications were made to the script: An error occurred (InvalidAMIID.NotFound) when calling the RunInstances operation. Run below script to get AMIID.

```
aws ec2 describe-images \

  --owners amazon \

  --filters "Name=name,Values=amzn2-ami-hvm-*-x86_64-gp2" \

  --query "Images | sort_by(@, &CreationDate) | [-1].ImageId" \

  --output text
```

Updated the subnet ID to deploy the instance into the private subnet.
Added the private subnet security group for restricted access.
Included tagging to simplify resource identification and management.
Additional Notes: Public subnets were configured with routing to an Internet Gateway. Private subnets were isolated to improve security. Security groups enforced instance-level access control. Connectivity testing validated successful network configuration.